Circuit ORAM: On Tightness of the Goldreich-Ostrovsky Lower Bound

Xiao Shaun Wang
wangxiao@cs.umd.edu
University of Maryland

T-H. Hubert Chan
hubert@cs.hku.hk
University of Hong Kong

Elaine Shi
runting@gmail.com
Cornell

ABSTRACT

We propose a new tree-based ORAM scheme called Circuit ORAM. Circuit ORAM makes both theoretical and practical contributions. From a theoretical perspective, Circuit ORAM shows that the well-known Goldreich-Ostrovsky logarithmic ORAM lower bound is tight under certain parameter ranges, for several performance metrics. Therefore, we are the first to give an answer to a theoretical challenge that remained open for the past twenty-seven years. Second, Circuit ORAM earns its name because it achieves (almost) optimal circuit size both in theory and in practice for realistic choices of block sizes. We demonstrate compelling practical performance and show that Circuit ORAM is an ideal candidate for secure multi-party computation applications.

Categories and Subject Descriptors

C.2.0 [**Computer-Communication Networks**]: General-security and protection

Keywords

Oblivious RAM; Secure Computation

1. INTRODUCTION

Oblivious RAM (ORAM), initially proposed by Goldreich and Ostrovsky [17, 19], is a general cryptographic primitive that allows oblivious accesses to sensitive data, such that access patterns during the computation reveal no secret information. Since the original proposal of ORAM [19], it has been studied in various application settings including secure processors [12,14,37,47], cloud outsourced storage [21,49,50, 57] and secure multi-party computation [15,16,25,28,33,53].

1.1 Rethinking ORAM Metric for Secure Computation

When ORAM was previously considered for cloud outsourcing and secure processor applications, its *bandwidth cost* [21, 48, 50–52] was used as a primary metric of performance, since it is well-understood that bandwidth is the main performance bottleneck in these scenarios. As a result, many existing ORAM schemes focused on optimizing the bandwidth metric [21, 48, 50–52].

With the new tree-based ORAM framework, it became feasible to implement ORAM atop secure multi-party computation (MPC) [25,28,53]. It is well-understood that ORAM carries the promise of scaling MPC to big data [33, 53].As a result, the community is interested in optimized ORAM constructions for MPC [15, 25, 28, 36, 53]. One recent revelation [53] is that ORAMs optimized for the bandwidth metric are not necessarily the best for the MPC scenario. Instead, MPC demands a new metric of ORAM schemes, namely, the *circuit complexity*. In a standard ORAM setting, a client interacts with a server multiple rounds to make an ORAM access, and performs computation in between these accesses. In an MPC scenario, the ORAM client's computation will be expressed as circuits that will be securely evaluated between the multiple parties. Therefore, an ORAM's circuit complexity is the total circuit size of the ORAM client algorithm over all rounds of interaction [53]. Furthermore, for several MPC protocols [18,58] where XOR operations are essentially free [30], the number of AND gates is the primary performance metric.

1.2 The Quest for ORAMs with Optimal Circuit Complexity

Due to increasing interest of implementing ORAMs to scale up MPC, the hunt is on for an ORAM scheme with optimal circuit complexity. Our new ORAM scheme, Circuit ORAM, is the first to give a compelling solution to this question in both practical and theoretical senses.

Compelling practical performance. In comparison with known ORAM schemes, Circuit ORAM achieves **58.4X** improvement in terms of number of non-free gates [30] over a straightforward implementation of Path ORAM [52], **31X** improvement over the binary-tree ORAM [48], and **5.1X** improvement over the recent SCORAM [53] – a heuristic ORAM scheme without theoretical performance bounds. These performance numbers are attained for a moderately large database of 4GB and with a 2^{-80} security failure probability. Our performance gains are asymptotic, so Circuit ORAM's improvement will be even greater for bigger data sizes.

Several earlier works on RAM-model MPC also investigated at what data sizes ORAM starts to outperform the trivial linear-scan ORAM, a metric referred to as the *breakeven point* with trivial ORAM. Previous implementations reported

CCS'15, October 12–16, 2015, Denver, Colorado, USA.

DOI: http://dx.doi.org/10.1145/2810103.2813634.

this breakeven point to be rather large. For example, Gordon *et al.* [25] who implemented the binary-tree ORAM [48] reported a breakeven point of 8MB with a block size of 512 bits and a security failure probability of roughly 2^{-14}. Under the same block size, we achieve an 8KB breakeven point (an **1000X** improvement) at a much tighter failure probability of 2^{-80}!

Theoretical near-optimality. For block sizes of $D = \Omega(\log^2 N)$ bits or higher, Circuit ORAM achieves a circuit size of $O(D \log N)\omega(1)$ gates for a $\mathsf{negl}(N)$ failure probability[1]. This is almost optimal since the well-known logarithmic ORAM lower bound [19] is immediately applicable to the circuit size metric as well. We discuss situations when the logarithmic ORAM lower bound [19] is tight in the following section.

Table 1 compares the circuit size of Circuit ORAM and existing works, both in terms of asymptotics and concrete performance numbers.

1.3 On Tightness of the Goldreich-Ostrovsky ORAM Lower Bound

For theoretical discussions regarding the tightness of the Goldreich-Ostrovsky ORAM lower bound, *we consider several additional metrics besides circuit size*, such as number of accesses and bandwidth blowup. We elaborate on these various metrics in Section 7.2.

In their seminal work, Goldreich and Ostrovsky [17, 19] showed a logarithmic lower bound for any ORAM construction. While not explicitly stated in their work, it is clear that this lower bound is very "powerful" in the sense that it holds *i)* for arbitrary block sizes, *ii)* for several relevant metrics including number of accesses, bandwidth blowup, and circuit size[2]; and *iii)* even when tolerating up to $O(1)$ statistical failure probability.

For mildly large block sizes of $\Omega(\log^2 N)$, Circuit ORAM achieves $O(D \log N + D \log \frac{1}{\delta})$ cost in terms of both *circuit size* and *bandwidth cost*, where δ is the failure probability. This means that for any $f(N) = \omega(1)$, there is an ORAM scheme that achieves $O(D \log N)f(N)$ circuit size or bandwidth cost, such that its statistical failure probability is bounded by some negligible function $\mathsf{negl}(N)$. In other words, for the circuit size or bandwidth cost metrics, **the Goldreich-Ostrovsky lower bound is asymptotically tight for $\Omega(\log^2 N)$ block size and negligible failure probabilities**. Equivalently, we rule out any $g(N)$ lower bound where $g(N) = \omega(\log N)$.

As we elaborate in Appendix 8 [3], we also show the tightness of the lower bound for the classical *runtime blowup* metric, but under a bigger block size of $\Omega(N^\epsilon)$ bits for an arbitrary constant $0 < \epsilon < 1$.

To summarize, one way to view our tightness result is the following: we give explicit and broad parameter ranges under which no asymptotically tighter lower bound can be proven. While this provides only a partial answer to the tightness of the Goldreich-Ostrovsky lower bound, we stress that for the past twenty-seven years, the tightness of the lower bound has not been demonstrated for any parameter ranges at all.

[1] Throughout this paper, the notation $g(N) = O(f(N))\omega(1)$ denotes that for any $\alpha(N) = \omega(1)$, $g(N) = O(f(N)\alpha(N))$.

[2] For number of accesses and bandwidth blowup, the lower bound is applicable to $O(1)$ client storage.

[3] Due to limit of space, all appendices are supplied in an online full version available at `https://eprint.iacr.org/2014/672.pdf`.

1.4 Technical Highlights

Path ORAM has a complex eviction circuit. In the secure processor setting, Path ORAM [52] and its improved variants [14, 16, 47] offer the best performance in terms of bandwidth cost. Therefore, the first natural idea is to try out Path ORAM for the MPC setting too. Unfortunately, Path ORAM's eviction circuit is complex, and would result in $O(D \log^2 N)$ size with a naive implementation. Although Wang *et al.* noted that Path ORAM's eviction algorithm can be implemented with a circuit of size $O(D \log N \log N \log N)\omega(1)$ using oblivious sorting [20], they also show that oblivious sorting makes the practical performance even worse than the naive $O(D \log^2 N)$ implementation for typical parameter ranges.

One thing to note is that Path ORAM's eviction algorithm in some sense performs (oblivious) sorting on $\Theta(\log N)$ items (imagine that bucket size were 1). Therefore, to do better we need a fundamentally different idea.

Reducing eviction complexity. Our idea is to find an eviction circuit that is less complex than that of Path ORAM's, and yet preserves the effectiveness of eviction. Achieving this is non-trivial. We first tested numerous ideas empirically, most of which failed to empirically bound the stash size since the eviction algorithm is not as aggressive as Path ORAM. After months of trying, we eventually identified a good empirical candidate, which in turn inspired the design of its provable variant, Circuit ORAM, that is documented in this paper.

Just like Path ORAM [52] and its variants [14, 16], Circuit ORAM performs eviction on $O(1)$ number of paths upon each data access. Our key idea is to complete the eviction algorithm within a single block scan of the current eviction path (while evicting as aggressively as we can). Achieving this directly introduces some difficulties due to a "lack-of-foresight" problem, i.e., the ORAM client does not know when to pick up a block and remove it from the path, and when to drop it into an empty slot on the path. To tackle this problem, we leverage two additional metadata scans to precompute the foresight required, before beginning the real block scan. Our construction and proofs share common themes with Path ORAM [52] and the CLP ORAM [7]. However, our construction and proofs *differ in a substantial and non-trivial manner from either Path ORAM or CLP ORAM.*

1.5 Related Work

Hierarchical ORAMs. Oblivious RAM was first proposed in a groundbreaking work by Goldreich and Ostrovsky [19]. In addition to the aforementioned lower bound, Goldreich and Ostrovsky were the first to propose a poly-logarithmic hierarchical construction, which was subsequently improved in numerous works [6, 17, 19, 21–24, 32, 35, 36, 42–44, 54–57]. Most of these hierarchical ORAM schemes are expensive in practice for MPC applications, not only due to their asymptotical poly-logarithmic cost (as opposed to logarithmic), but also crucially, because the ORAM client in these schemes must compute a PRF function – in an MPC application, this PRF would have to be securely evaluated using a multi-party protocol. Further, all schemes dependent on

Scheme	Circuit Size (asymptotic)*	# AND gates (concrete)**
Hierarchical ORAMs		
GO96 [17, 19]	$O(D \log^3 N + C_{\text{PRF}} \log^2 N)$	≥476.1M
GM11 [21]	$O(D \log^2 N + C_{\text{PRF}} \log N)$	
KLO12 [32]	$O((D + C_{\text{PRF}}) \cdot \log^2 N / \log \log N)$	≥linear scan
LO13 [36] (Note: 2-server model)	$O((D + C_{\text{PRF}}) \cdot \log N)$	(63488M)
Tree-based ORAMs		
Binary-tree ORAM [48]	$O((D + \log^2 N) \log^2 N)\omega(1)$	30.1M
CLP13 [7] (naive circuit)	$O((D + \log^2 N) \log^3 N)\omega(1)$	37.9M
CLP13 [7] (w/ oblivious queue [39, 45, 59])	$O((D + \log^2 N) \log^2 N)\omega(1)$	37.9M
Path ORAM (naive circuit) [52]	$O((D + \log^2 N) \log^2 N)\omega(1)$	56.6M
Path ORAM (o-sort circuit) [53]	$O((D + \log^2 N) \log N \log \log N)\omega(1)$	41.4M
Circuit ORAM (This Paper)	$\mathbf{O((D + \log^2 N) \log N)\omega(1)}$	**0.97M**

Table 1: **Circuit size of various ORAM schemes.** D is the bit length of each block, and N is the total number of blocks. All schemes are parameterized to have $\frac{1}{N^{\omega(1)}}$ failure probability.
*: The variable C_{PRF} denotes the circuit size of a PRF function with input size of $O(\log N)$ bits. Among all single-server ORAM schemes, Circuit ORAM has asymptotically the smallest circuit size if C_{PRF} is at least $\omega(\log N \log \log N)$ — which is true for all known PRF constructions provably secure based on computationally hard problems.
**: The concrete circuit size is calculated based on 4GB data with a 32-bit block size, with 2^{-80} security failure probability.

cuckoo hashing [21, 32, 36] (including the two-server ORAM by Lu and Ostrovsky [36]) require the smallest level to have sizeΩ $(\log^7 N)$ to tightly bound the failure probability of cuckoo hashing. Technically, this means that these schemes basically reduce to the trivial ORAM (or the Goldreich-Ostrovsky ORAM [17, 19]) for $N < 2^{37}$.

Tree-based ORAM framework. The tree-based ORAM framework, initially proposed by Shi *et al.* [48], departs fundamentally from the hierarchical framework [19], and is a new paradigm for constructing a class of ORAM schemes. Several later works [7,15,52] improved Shi *et al.*'s initial construction [48] (commonly referred to as binary-tree ORAM). These schemes are conceptually simpler, statistically secure, and easy to implement in secure processors [12, 14, 37, 47] or MPC applications [16, 25, 28, 33, 53]. A more detailed description of tree-based ORAMs are provided in Section 2.

Remarks about the ORAM lower bound. Besides efforts at constructing more efficient upper bounds, the community has also been interested in tightening the lower bound. Beame and Machmouchi [5] show a super-logarithmic lower bound for oblivious branching programs. Although some works cited their lower-bound as being applicable to the ORAM setting [8, 21], it was later recognized that **Beame *et al.*'s super-logarithmic lower bound is not applicable to the standard model of Oblivious ORAM.** As the authors noted themselves in an updated version [5], one key difference is that the standard ORAM model requires that the probability distribution of the observed access patterns be statistically close regardless of the input; whereas Beame's model requires that for each given random string r, the access pattern be independent of the input. Therefore, their super-logarithmic lower-bound is in a much stronger model than standard ORAM, and hence inapplicable to ORAM. To date, Goldreich and Ostrovsky's original lower bound is still the best we know.

Oblivious storage and server-side computation. The original ORAM model proposed by Goldreich and Ostrovsky assumes a passive server (or memory) that does not perform computation. However, several subsequent works leveraged server-side computation to improve performance [9, 10, 38, 46, 57] or reduce the number of roundtrips [56].

To distinguish this server-computation setting from standard ORAM, we refer to it as *oblivious storage* as suggested by several earlier works [3, 6]. Apon *et al.* [3] point out that the *Goldreich-Ostrovsky lower bound is not applicable to the oblivious storage setting for the bandwidth metric.* In fact, one can construct oblivious storage schemes with constant bandwidth blowup but with poly-logarithmic server work. [10].

Most of existing oblivious storage schemes (with server computation) are unsuitable for the MPC setting, because they focus on optimizing the client-server bandwidth while paying the price of higher, typically poly-logarithmic server work. In an MPC setting, all server-side work must be securely evaluated using an MPC protocol which immediately incurs poly-logarithmic circuit size, and is thus expensive.

Subsequent work. Circuit ORAM is now provided as the default ORAM implementation in the ObliVM secure computation framework by Liu *et al.* [34]. ObliVM is based on garbled circuits, and at the front-end provides expressive programming abstractions and language features for non-specialist programmers. Using Liu *et al.*'s ObliVM framework, in Section 5.1 we provide more detailed end-to-end performance of Circuit ORAM.

2. PRELIMINARIES

Definitions. Our definition of Oblivious RAM (ORAM) is standard, and we therefore defer formal definitions to Appendix 7.1 [1]. Below, we introduce the tree-based ORAM framework.

2.1 Tree-based ORAM Framework

Shi *et al.* proposed a new tree-based framework [48], which was adopted subsequently by several improved con-

structions [7,15,40,52,53]. We now briefly review the framework.

Notation. We use N to denote the number of (real) data blocks in ORAM, D to denote the bit-length of a block in ORAM, Z to denote the capacity of each bucket in the ORAM tree, and λ to denote the ORAM's statistical security parameter. When discussing binary trees of depth $L = \log N + 1$ in this paper, we say the *leaves* are at level L and the root is at level 1. For convenience in algorithm descriptions, we sometimes treat the stash as a depth-0 bucket with some capacity R that is the *imaginary parent* of the root. We assume that leaves are numbered sequentially from 0 to $N-1$. We also denote $[a..b] := \{a, a+1, \ldots, b\}$.

Data structure. The server organizes *blocks* into a binary tree of height $L = \log N + 1$; each node of the tree is a *bucket* containing Z blocks. Each block is of the form:

$$\{\texttt{idx}||\texttt{label}||\texttt{data}\},$$

where `idx` is the index of a block, e.g., the (logical) address of desired block; `label` is a leaf identifier specifying the path on which the block resides; and `data` is the payload of the block, of D bits in size.

The client stores a *stash* for buffering overflowing blocks. In certain schemes such as the original binary-tree scheme [48], such a stash is not necessary. In this case, we can simply treat this as a degenerate stash of size 0.

The client also stores a *position map*, mapping a block's `idx` to a leaf `label`. As described later, position map storage can be reduced to $O(1)$ by recursively storing the position map in a smaller ORAM. These leaf labels are assigned randomly and are reassigned as blocks are accessed. If we label the leaves from 0 to $N-1$, then each label is associated with a path from the root to the corresponding leaf.

Main path invariant. Tree-based ORAMs maintain the invariant that a block marked label resides on the path from the stash (to the root) to the leaf node marked label.

Operations. Tree-based ORAMs all follow a similar recipe as shown in Figure 1. In particular, the ReadAndRm operation would read every block on the path leading to the leaf node marked `label`, and fetches and removes the block `idx` from the path.

Various tree-based ORAMs are differentiated by the eviction algorithm denoted Evict(). For example, the original binary-tree ORAM adopts a simple eviction algorithm engineered to make their proof easy: with each data access, two distinct buckets are chosen at random from each level to evict from. By contrast, the Path ORAM algorithm performs eviction on the read path, and the eviction strategy is aggressive: pack all blocks as close to the leaf as possible respecting the main invariant. In Path ORAM, a $O(\log N) \cdot \omega(1)$ stash is necessary to buffer overflowing blocks.

Recursion. Instead of storing the entire position map in the client's local memory, the client can store it in a smaller ORAM on the server. In particular, this position map ORAM needs to store N labels each of $\log N$ bits. We can apply this idea recursively until we get down to a constant amount of metadata, which the client could store locally.

As mentioned in Appendix 8 [1], we will leverage the "big block, little block" trick first proposed by Stefanov *et al.* [52] to parametrization the recursion, such that the recursion does not introduce additional asymptotic cost in terms of circuit size.

3. Circuit ORAM

3.1 Overview

Circuit ORAM follows the tree-based ORAM framework, by building a binary tree containing N nodes (referred to as *buckets*), where each bucket can store $Z = O(1)$ number of blocks.

Stash. As later proved in Theorem 1 and 2, with probability at least $1 - 2^{-\Omega(R)}$, the stash holds at most R blocks. We can parameterize $R = O(\log N) \cdot \omega(1)$ to obtain a failure probability negligible in N. To achieve $O(D)$ bits of client space, **this stash can be stored on the server side**, and operated on by the client in each data access obliviously. For convenience, we will often refer to the stash as being the 0-th level on the path, i.e., path[0].

Operations. The data access algorithm `Access` follows the same structure as in the binary-tree ORAM [48] or Path ORAM [52] – explained in Figure 1 in Section 2. It suffices for us to describe how eviction is implemented in Circuit ORAM, which we will focus on in the remainder of this section.

Definition 1 (Legally reside). *We say that a block* B *can legally reside in* path[ℓ] *if by placing* B *in* path[ℓ]*, the main path invariant is satisfied.*

Definition 2 (Deepness w.r.t eviction path). *For a given eviction path denoted* path*, block* B_0 *is deeper than block* B_1 *(with respect to* path*), if there exists some* path[ℓ] *such that* B_0 *can legally reside in* path[ℓ]*, but* B_1 *cannot; in the case when both blocks can legally reside in the same buckets along* path*, the block with smaller index* `idx` *will be considered deeper.*

In other words, B_0 is deeper on the current eviction path than B_1 if it can legally reside nearer to the leaf along path. If two blocks have the same deepness, we use their indices `idx` to resolve ambiguity. This will be useful later in our proofs.

Our notion of deepness and the greedy eviction choice of the deepest block on a path are inspired by the novel ideas of the CLP ORAM [7] – but it will soon become apparent that we apply it in a fundamentally different manner.

3.2 Intuition

We would like to have an eviction algorithm that is easy to implement as a small circuit. Ideally it should make *a single scan* of the data blocks on the eviction path from the stash to leaf (and only a constant number of metadata scans), and still try to push blocks towards the leaf as much as possible.

During the one-pass scan of the data blocks, we would like the client to "pick up" (i.e., remove from path) and hold onto one block, which can later be "dropped" somewhere further along the path. At any point of time, the client should hold onto at most one block. Further, it makes sense for the client to hold onto the currently *deepest* block when it does decide to hold a block. This way, the block in holding will have the maximum chance of being dropped later. On encountering a deeper block, the client could swap it with the one in holding.

Figure 1: Access(op) // where op = ("read", idx) or op = ("write", idx, data*)

1: label := PositionMap[idx]
2: {idx||label||data} := ReadAndRm(idx, label)
3: PositionMap[idx] := UniformRandom(0 ... N − 1)
4: If op is "read": data* := data
5: stash.add({idx||PositionMap[idx]||data*})
6: Evict()
7: Return data

However, a dilemma arises. How does the client decide when it should pick up a block and hold onto it? Maybe this block will never get a chance to be dropped later, in which case there will be two equally bad choices: 1) put the block into the stash – which results in rapid stash growth; and 2) go back and revisit the path to write the block back. However, doing this obliviously results in high cost.

Remedy: lookahead mechanism with two metadata scans. The above issues result from the lack of foresight. If the client could only know when to pick up a block and place it in holding, and when to write the block back into an available slot, then these issues would have been resolved. Our idea, therefore, is to rely on two metadata scans prior to the real block scan, to compute all the information necessary for the client to develop this foresight. These metadata scans need not touch the actual blocks on the eviction path, but only metadata information such as the leaf label for each block, and the dummy bit indicator for each block. If the bucket size is $O(1)$, then the bandwidth blowup is $O(\log N)$. The most technical part of the proof is to show that the stash size is still $O(\log N)$ with similar failure probability as Path ORAM.

3.3 Detailed Scheme Description

To describe a circuit representation of our eviction algorithm, we write it as an *oblivious* algorithm. A program is oblivious if its memory accesses do not depend on secret inputs. In other words, all memory addresses can be statically inferred from public values. When we construct a circuit, the circuit's wiring must be statically inferred (as opposed to at runtime). This is exactly why an oblivious (and deterministic program) can easily be encoded as a circuit.

A slow and non-oblivious version of the eviction algorithm. To aid understanding, we first describe a slow, non-oblivious version of our eviction algorithm, EvictOnceSlow, as shown in Algorithm 1. This slow version only serves to illustrate the effect of the eviction algorithm, but does not describe how the algorithm can be efficiently implemented in circuit. Furthermore, this slow, non-oblivious version of our eviction algorithm gives a simpler way to reason about the stash usage of the algorithm, and hence will facilitate our proofs later. Later in this section, we describe how to implement our eviction algorithm efficiently and obliviously by making use of two metadata scans and a one real block scan; this can be readily converted into a small-sized circuit.

The EvictOnceSlow algorithm makes a reverse (i.e., leaf to stash) scan over the current eviction path. When it first encounters an empty slot in path[i], it will try to evict the deepest block B in path[$0..i-1$] to this empty slot, provided that the block B can legally reside in path[i]. Suppose this deepest block B resides in path[ℓ] where $\ell < i$. After relocating the block B to path[i], the algorithm now skips levels path[$\ell+1..i-1$], and continues its reverse scan at level ℓ instead (Line 8 in Algorithm 1). In case no block in path[$0..i-1$] can fill the empty slot in path[i], the scan simply continues to level path[$i-1$].

Efficient and oblivious implementation of our eviction algorithm. In Algorithm 1, Line 4 is inefficient, and Line 8 is non-oblivious. We now explain how to implement the same EvictSlow algorithm obliviously and efficiently, but using two metadata scans (Algorithms 2 and 3) plus a single real block scan (Algorithm 4). Since metadata is typically much smaller than real data blocks, a metadata scan is faster than a real block scan.

The two metadata scans will generate two helper data structures:

- An array deepest[$1..L$], where deepest[i] = ℓ means that the deepest block in path[$0..i-1$] that can legally reside in path[i] is now in level $\ell < i$. If no block in path[$0..i-1$] can legally reside in path[i], then deepest[i] := $\perp$. In the pre-processing state, we will use one metadata scan, namely the PrepareDeepest subroutine (see Algorithm 2), to populate the deepest array. This allows us to avoid Line 4 in Algorithm 1 causing an additional $\Theta(L)$ overhead.
- An array target[$0..L$], where target[i] stores which level the deepest block in path[i] will be evicted to. This target array is prepopulated using a backward metadata scan as depicted in the PrepareTarget algorithm (see Algorithm 3).

Observe that the prepopulated target array basically gives a precise prescription of the client's actions (including when to pick up a block and when to drop it) during the real block scan. At this moment, the client performs a forward block scan from stash to leaf, as depicted in the EvictOnceFast algorithm (see Algorithm 4). The high level idea here is to "hold a block in one's hand" as one scans through the path, where the block-in-hand is denoted as hold in the algorithm. This block hold will later be written to its appropriate destination level, when the scan reaches that level.

Example. To aid understanding, a detailed example, including the PrepareDeepest, PrepareTarget, and the EvictOnceFast steps, is given in Figure 2.

Eviction rate and choice of eviction path. For each data access, two paths are chosen for eviction using the EvictOnceFast algorithm. While other approaches are conceivable, we describe two simple ways for choosing the eviction paths:

- A *random-order* eviction strategy denoted EvictRandom() (see Algorithm 5). The randomized strategy chooses two random paths that are non-overlapping except at

Algorithm 1 `EvictOnceSlow(path)`
*/*A slow, non-oblivious version of our eviction algorithm, only for illustration purpose*/*

```
1: i := L        /* start from leaf */
2: while i ≥ 1 do:
3:     if path[i] has empty slot then
4:         (B, ℓ) := Deepest block in path[0..i − 1] that can legally reside in path[i].
               /* B := ⊥ if such a block does not exist.*/
5:     end if
6:     if B ≠ ⊥ then
7:         Move B from path[ℓ] to path[i].
8:         i := ℓ      // skip to level ℓ
9:     else
10:        i := i − 1
11:    end if
12: end while
```

Algorithm 2 `PrepareDeepest(path)`
*/*Make a root-to-leaf linear metadata scan to prepare the* deepest *array.*
After this algorithm, deepest[i] *stores the source level of the deepest block in* path[$0..i-1$] *that can legally reside in* path[i]. **/*

```
1: Initialize deepest := (⊥, ⊥, ..., ⊥), src := ⊥, goal := −1.
2: if stash not empty then
       src := 0,
       goal := Deepest level that a block in path[0] can legally reside on path.
3: end if
4: for i = 1 to L do:
5:     if goal ≥ i then deepest[i] := src
6:     end if
7:     ℓ := Deepest level that a block in path[i] can legally reside on path.
8:     if ℓ > goal then
9:         goal := ℓ, src := i
10:    end if
11: end for
```

the stash and the root. This means that one path is randomly chosen from each of the left and the right branches of the root.

- A *deterministic-order* strategy denoted as `EvictDeterministic()` (Algorithm 6). This is inspired by Gentry *et al.* [15] and several subsequent works [13, 16].

Recursion. So far, we have assumed that the client stores the entire position map. Based on a standard trick [48, 51, 52], we can recursively store the position map on the server. In the position map recursion levels, we will use a different block size than the main data level as suggested by Stefanov *et al.* [52]. Specifically, we group c number of `labels` in one block for an appropriate constant $c > 1$. In other words, the block size for position map levels is set to be $D' = O(\log N)$, resulting in $O(\log N)$ depth of recursion. In this way, our total bandwidth cost over all recursion levels would be $O(D \log N + \log^3 N) \cdot \omega(1)$ (for negligible failure probability), assuming that the stashes reside on the server side, and the hence client only needs to hold a constant number of blocks at any time. For inverse polynomial failure probability, the total bandwidth cost is $O(D \log N + \log^3 N)$.

Security proof. The security proof is trivial. First, as in all tree-based ORAMs, every time a block is read or written, a random path is read, where the random choice has not been revealed to the server before. This part of the proof is trivial, and the same as Shi *et al.* [48]. We now show that the eviction process is oblivious too. As we can see from Algorithms 2,3 and 4, eviction on a selected path always reads blocks or metadata (stored on the server) in a sequential manner, either from leaf to root or from root to leaf. Clearly this does not depend on the logical address being read or written. In fact, saying that our eviction algorithm (Algorithms 4) is oblivious is the same as saying that it can be implemented efficiently in circuit representation! Finally, no matter whether we use random-order eviction (Algorithm 5) or deterministic order eviction (Algorithm 6), the choice of the eviction path is also independent of the logical address sequence being read/written.

3.4 Theoretical Bounds

Although our scheme superficially borrows the "deepest" idea from the CLP ORAM, and borrows the "eviction on a path" idea from Path ORAM, *we stress that our construction is fundamentally different from either which necessitates novel proof techniques.*

A slightly modified Circuit ORAM construction. For subtle technical reasons, in our proofs we need to make a minor modification to the main construction. Since this modification is not very interesting, we did not document it in our main scheme for clarity. The modification involves introducing an additional *partial eviction* performed on the read path, simply to fill up the hole that is newly created by the

Algorithm 3 `PrepareTarget(path)`
*/*Make a leaf-to-root linear metadata scan to prepare the* target *array. */*
After this algorithm, if target$[i] \neq \perp$*, then one block shall be moved from* path$[i]$ *to* path[target$[i]$] *in* `EvictOnceFast(path)`*. */*

```
1: dest := ⊥, src := ⊥, target := (⊥, ⊥, ..., ⊥)
2: for i = L downto 0 do:
3:     if i == src then
4:         target[i] := dest, dest := ⊥, src := ⊥
5:     end if
6:     if ((dest = ⊥ and path[i] has empty slot) or (target[i] ≠ ⊥)) and (deepest[i] ≠ ⊥) then
7:         src := deepest[i]
8:             /* deepest is populated earlier using the PrepareDeepest algorithm.*/
9:         dest := i
10:    end if
11: end for
```

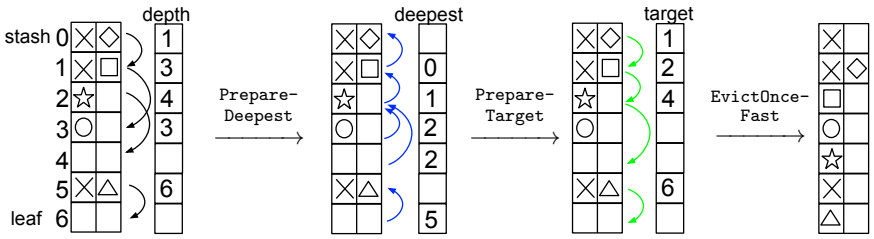


Figure 2: **An example of Circuit ORAM eviction.** Consider an eviction path with the stash at level 0, and the leaf at level 6. Bucket size and stash size are both 2. When there are two blocks in a level, $\times$ represents a block that is the *less deep*. The eviction will not care about the $\times$ blocks, but all other blocks are of potential interest, so we use distinct shapes to distinguish them.
At the beginning, `depth[i]` contains the deepest level of blocks in level i. The results of the two metadata scans are stored in the arrays `deepest` and `target` respectively. To aid understanding, we use arrows to visualize the arrays `depth`, `deepest`, and `target`. More detailed explanations of the arrows in each subfigure are provided below.
(a) $s \to t$: A black arrow from level s to level t means the following: a block in path$[s]$ can legally reside in path$[t]$; but no block in path$[s]$ can legally reside in path$[t+1..L]$. Here $s < t$.
(b) $s \to t$: A blue arrow from level s to level t means the following: the deepest block in path$[0..s-1]$ that can legally reside in path$[s]$ currently resides in path$[t]$. Here $t < s$.
(c) $s \to t$: A green arrow from level s to level t means the following: during the real block scan, the client should pick up the deepest block in path$[s]$, and drop it in path$[t]$. Here $s < t$.
(d) Blocks are evicted according to target pointers (in green).

ReadAndRm operation. A partial eviction works just like a normal eviction, but works on only part of the path upto the point where the the block is removed (and for obliviousness dummy eviction operations are performed for the rest of the path). We refer the readers to Appendix 9 [1] for a detailed description of the modification. This additional modified partial eviction is only necessary to to show an equivalence between a post-processed ∞-ORAM and the real ORAM (see Section 4 and Appendix 9 for more details).

In our experiments described in Section 5, we also chose not to implement this partial eviction — this leads to slightly better empirical results. However, even if one chooses to implement this partial eviction in practice, it would only incur a small constant factor penalty.

Stash bounds. We prove stash bounds for both random-order and deterministic-order eviction. Below we give the formal theorem statements but defer their proofs to the appendices.

THEOREM 1 (STASH GROWTH FOR RANDOM EVICTION). *Let the bucket size $Z \geq 5$. Let* st(ORAM^Z[s]) *be a random variable denoting the stash size after access sequence* s *for a Circuit ORAM with bucket size Z and randomized eviction. Then, for any access sequence* s*,*

$$\Pr\left[\mathsf{st}(\mathsf{ORAM}^Z[\mathbf{s}]) > R\right] \leq 42 \cdot 0.6^R$$

where probability is taken over the ORAM algorithm's randomness.

The detailed proof of this theorem is deferred to Appendix 9 and 10.1. .

THEOREM 2. (STASH GROWTH FOR DETERMINISTIC EVICTION). *Let the bucket size $Z \geq 4$. Let* st(ORAM^Z[s]) *be a random variable denoting the stash size after access sequence* s *for a Circuit ORAM with bucket size Z and deterministic eviction. Then, for any access sequence* s*,*

$$\Pr\left[\mathsf{st}(\mathsf{ORAM}^Z[\mathbf{s}]) > R\right] \leq 14 \cdot e^{-R},$$

where probability is taken over the ORAM algorithm's randomness.

Algorithm 4 `EvictOnceFast(path)`

Call the `PrepareDeepest` and `PrepareTarget` subroutines to pre-process arrays deepest and target.
hold := $\perp$, dest := $\perp$.
for $i = 0$ to L **do**
    towrite := $\perp$
    if (hold $\neq \perp$) and ($i ==$ dest) **then**
        / The block stored in* hold *will be placed in bucket* path$[i]$*. */*
        towrite := hold
        hold := $\perp$, dest := $\perp$.
    end if
    if target$[i] \neq \perp$ **then**
        hold := read and remove deepest block in path$[i]$
        dest := target$[i]$
    end if
    Place towrite into bucket path$[i]$ if towrite $\neq \perp$.
end for

Algorithm 5 `EvictRandom()`

Choose a leaf from each of the left and the right branches of the root independently, and denote the two corresponding (stash-to-leaf) paths by path_0 and path_1.
Call EvictOnceFast(path_0) and EvictOnceFast(path_1)

Algorithm 6 `EvictDeterministic()`

In timestep t:
Choose two paths, path_0 and path_1, corresponding to the leaves labeled with integers bitrev($2t \mod N$) and bitrev($(2t+1) \mod N$), respectively. In the above bitrev(i) denotes the integer obtained by reversing the bit order of i when expressed in binary.
Call EvictOnceFast(path_0) and EvictOnceFast(path_1)
Increase t by 1 for the next access.

The detailed proof of this theorem is deferred to Appendix 9 and 10.2 [1]. .

While our formal proof requires $Z \geq 5$ for randomized eviction and $Z \geq 4$ for deterministic-order eviction, empirical results show that choosing $Z = 3$ for randomized eviction and $Z = 2$ for deterministic eviction would result in bounded stash size R with failure probability $2^{-\Theta(R)}$.

THEOREM 3 (CIRCUIT SIZE BOUND). *Circuit ORAM achieves* $O((D+\log^2 N)(\log N+\log \frac{1}{\delta}))$ *circuit size for a statistical failure probability of* δ*. Specifically, if the block size* $D = \Omega(\log^2 N)$*, then Circuit ORAM achieves* $O(D(\log N + \log \frac{1}{\delta}))$ *circuit size for a statistical failure probability of* δ*.*

PROOF. For $D = \Omega(\log^2 N)$, consider Circuit ORAM with big data blocks of $O(D)$ bits, and little metadata blocks of $O(\log N)$ bits (used in the position map levels of the recursion). In Theorems 1 and 2, we show that the stash size is $O(\log \frac{1}{\delta})$ blocks for a failure probability of δ. The rest immediately follows. □

4. PROOF ROADMAP

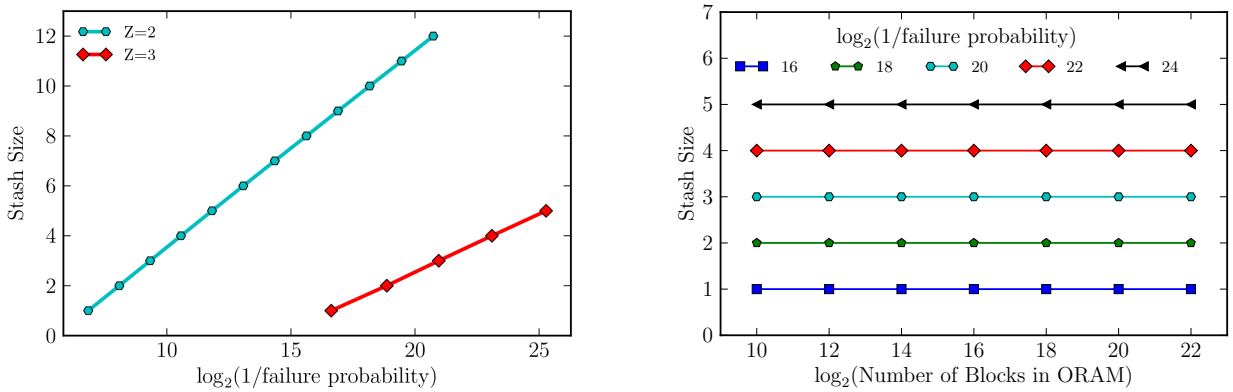
We give a roadmap of how the probability statements concerning stash usage in Theorems 1 and 2 are proved. The full proofs are given in Appendices 9 and 10 [1]. Although our proof borrows some high-level ideas from both Path ORAM [52] and CLP ORAM [7], we stress that *both our construction and proof are non-trivial and differ from Path ORAM and CLP ORAM in significant ways.*

Equivalence to post-processed ∞-ORAM. Similar to Path ORAM [52]'s analysis, we consider an imaginary construct known as ∞-ORAM, which is the same as Circuit ORAM, except that buckets have infinite capacity. ∞-ORAM is not a real-world efficient ORAM scheme, but a construct that facilitates analysis.

Suppose we are given the state of the ∞-ORAM at some moment, and we would like to infer from it the stash usage of the real ORAM. One natural way is to post-process the ∞-ORAM such that if a bucket contains more blocks than its capacity, the extra blocks are pushed back to its parent. This is performed repeatedly until all extra blocks are pushed from the root to the stash. As in the analysis of Path ORAM, the hope is that the stash usage of the post-processed ∞-ORAM is the same as that of the real ORAM. If this is true, then we can use the same approach [52] to analyze ∞-ORAM.

It is intuitive that the stash usage of the real ORAM should be at least that of the post-processed ∞-ORAM. Our ∞-ORAM shows how far blocks could be evicted towards the leaves even without any restrictions because of bucket capacity. The post-processing adds back the bucket capacity requirement. Hence, the post-processed ∞-ORAM gives some bound on how far the blocks could be evicted towards the leaves in the real ORAM.

However, it is not obvious that the real ORAM could evict blocks towards the leaves to the same extent as the post-processed ∞-ORAM. Indeed, if we do not perform partial eviction on the read path in a ReadAndRm operation (see Appendix 9.1), then a hole (an available slot that would be



(a) **The stash exceeds R with probability $2^{-\Theta(R)}$.** $N = 2^{10}$ is used.

(b) **The stash size is independent of the ORAM capacity N.** $Z = 3$ is used.

Figure 3: **Evaluation of stash size.** Deterministic-order eviction is adopted.

filled in post-processing) could be created in the real ORAM. This could mean an extra block has to be stored in the stash of the real ORAM, thereby breaking the equivalence of stash usage between the real ORAM and the post-processed ∞-ORAM.

Fortunately, a partial eviction in a ReadAndRm operation is sufficient to fix this issue. However, unlinke Path ORAM's analysis, in our case it is highly non-trivial to show that indeed the post-processed ∞-ORAM is equivalent to the real Circuit ORAM. The key insight is to maintain the invariant (Fact 2 in Appendix 9) that if some bucket in the real ORAM has an available slot, then during the post-processing of ∞-ORAM, no blocks can be pushed through this bucket towards the root. However, to formalize this argument requires an intricate induction proof that is presented in Appendix 9. This technical proof can be skimmed, if the reader is convinced of the validity of the post-processed ∞-ORAM.

Probabilistic tools to analyze ∞-ORAM. Once it is established that the post-processed ∞-ORAM is equivalent to the real ORAM as in the analysis of Path ORAM [52], one can observe that the post-processed ∞-ORAM has stash usage of R blocks *iff* there exists a subtree T at the root with $n := n(T)$ buckets in ∞-ORAM such that the number of blocks residing in T is $nZ + R$, where Z is the bucket capacity. The goal is to show that at some fixed moment, for a fixed subtree T, the probability that T (in unprocessed ∞-ORAM) contains at least $nZ+R$ blocks is at most $\exp(-Cn-R)$, for some large enough constant $C > 0$. Since there are at most 4^n subtrees with n nodes, a union bound over all possible subtrees T can establish the probability bound in Theorems 1 and 2.

The full proof is in Appendix 10 [1], and we outline the key ideas here. To analyze the usage of the subtree T, we consider two cases at the subtree's boundary, where blocks might possibly be evicted from T.

- Suppose bucket u in T is also a leaf in ∞-ORAM, and let X_u be the number of blocks in T whose label corresponds to u. Observe that these blocks cannot leave T. Since there are N distinct blocks, by a standard balls-into-bins argument, in the worst case, X_u is a sum of N independent $\{0,1\}$-random variables each having mean $\frac{1}{N}$.
- Suppose bucket u is not in T, but its parent bucket is in T. In this case, we say that u is an *exit* node, and let X_u be the number of blocks in T that can legally reside in u. Since in each ORAM access operation, a block is assigned a fresh random label and there are two eviction paths, we shall argue that X_u can be viewed as a Markov queue whose departure rate is twice that of its arrival rate. For random eviction path selection, X_u behaves like a a discrete-time M/M/1 queue, while for deterministic-order eviction variant, X_u behaves like a discrete-time M/D/1 queue [29]. Assuming that Circuit ORAM is initially empty, we can instead consider the stochastically dominating scenario when X_u is already in the stationary distribution, whose analysis does not depend on the number N of distinct blocks.

We shall see that in either of the above cases, the random variable X_u has constant expectation. Hence, the number of blocks in T, which is the sum of the X_u's, has expectation $\Theta(n)$. Observe that in the analysis of CLP ORAM [7], they consider how often each X_u reaches some threshold, whereas we directly consider the sum of the X_u's as in [52]. We next use a measure concentration argument to prove that the probability that the sum deviates from its mean is exponentially small.

Observe that the X_u's are not independent. In fact, the X_u's are *negatively associated* [11], because if a block is assigned to one of the X_u's, then it cannot be assigned to another. Nevertheless, negative associativity is enough to prove measure concentration results via moment generating functions, which are standard tools used to derive results like Chernoff and Hoeffding bounds.

In Appendix 10, we formally prove that the X_u's are negative associated using Lemma 10, and give upper bounds for the moment generating functions $t \mapsto \mathbb{E}[\exp(tX_u)]$ of the X_u's in Lemma 9. Finally, the probability calculations to achieve Theorems 1 and 2 are completed in the proof of Lemma 8 [1].

Type Of ORAM	Circuit ORAM		Path ORAM(naive)		Path ORAM(o-sort)		SCORAM	
	Det.	Rand.	Det.	Rand.	Det.	Rand.	Det.	Rand.
Circuit size	$3.5M$	$6.6M$	$28.5M$	$170.1M$	$62.1M$	$124.1M$	$14.5M$	$17.9M$
Relative overhead	$1X$	$1.9X$	$8.2X$	$48.6X$	$17.7X$	$35.5X$	$4.14X$	$5.1X$
#AND gates	$0.97M$	$1.6M$	$9.5M$	$56.6M$	$20.7M$	$41.4M$	$4.9M$	$6.1M$
Relative overhead	$1X$	$1.6X$	$9.79X$	$58.4X$	$21.34X$	$42.7X$	$5.1X$	$6.3X$

Table 2: **Comparison of Circuit ORAM and variant of Path ORAM.** $N = 2^{30}, D = 32, \delta = 2^{-80}$. Path ORAM(o-sort) [53] uses 3 o-sorts. "Rand." stands for randomly chose eviction paths; "Det." stands for eviction with reverse-lexicographical-ordered paths, described in earlier works [13, 15, 16].

		Circuit ORAM $\delta = 2^{-80}$					GKKKMRV12 [25] $\delta \approx 2^{-14}$	KS12 [28] $\delta = 2^{-20}$
Block size (bits)	32	**40**	128	**512**	2048	8192	512	40
Breakeven point (entries)	256	**256**	128	**128**	64	64	131072	≈2000
Breakeven point (total data size)	1KB	**1.3KB**	2KB	**8KB**	16KB	65KB	8MB	9.77KB

Table 3: **Breakeven point for different ORAMs.** Circuit ORAM numbers correspond to a security parameter of 80, whereas GKKKMRV12 and KS12 adopt a security parameter of roughly 14 and 20 respectively.

5. EVALUATION

Stash size distribution. We simulate Circuit ORAM for a single long run, for about 2^{33} accesses, after 2^{25} accesses to warm up the ORAM to steady state. In our experiments, we use the following request sequence where we repeatedly cycle through all N logical addresses: $1, 2, \ldots, N, 1, 2, \ldots, N, \ldots$. Using the same argument as in Path ORAM [52], it is not hard to see that this is the worst-case access sequence. Instead of measuring multiple runs, we use a single, very long run, and measure the fraction of time that the stash has a certain size. This methodology is well-founded, since it is well-known that if a stochastic process is regenerative, the time average over a single run is equivalent to the ensemble average over multiple runs (see Chapter 5 of Harchol-Balter [26]).

Figure 3a plots the stash size against the quantity $\log(\frac{1}{\delta})$ where δ is the failure probability. A point on the curve should be interpreted as: the stash exceeds R (value on y-axis) with probability δ. The two curves correspond to a bucket size of 2 and 3 respectively. Clearly, the fact that both curves are a linear line suggests that the stash exceeds R with probability of 2^{-cR}, where the constant c in the exponent is different for the two curves. Further, Figure 3b shows that the stash size is independent of the ORAM's capacity N. Besides a bucket size of 2 and 3, we also tried a bucket size of 4 – in this case, we never observed the stash growing beyond 5 for the first 2^{33} accesses.

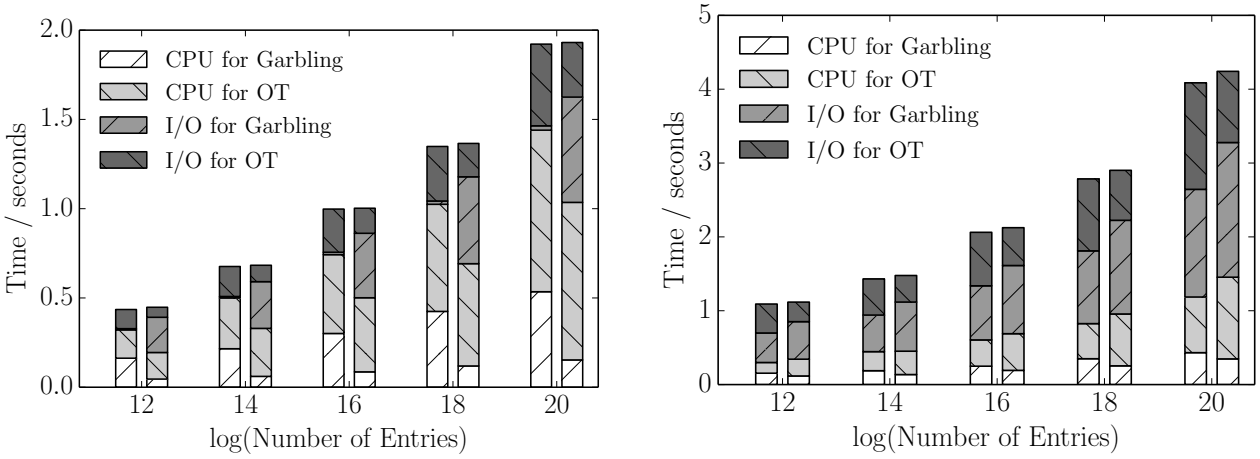
Circuit size. In Table 2, we compare the circuit sizes of Circuit ORAM and other state-of-the-art ORAM schemes. Results in this table are obtained for a 4GB dataset with the following concrete parameters: $N = 2^{30}$, $D = 32$ bits, and security failure $\delta = 2^{-80}$. For Path ORAM [52], we consider a naive implementation, and an asymptotically more efficient implementation relying on 3 oblivious sorts [53]. For all schemes, we consider two strategies for choosing the eviction path: random-order eviction and deterministic-order eviction (based on *digit-reversed lexicographic order* [15]). The table shows that Circuit ORAM results in 8.2x to 48.6x smaller circuit size than Path ORAM, and is 4.1x to 5.1x better than SCORAM [53]. Circuit ORAM's speedup will become even bigger when the total data size N is greater.

Breakeven point with trivial ORAM. Depending on the block size, the breakeven point between Circuit ORAM and trivial ORAM varies – Table 3. We also compare our breakeven point with Gordon *et al.* [25], and Keller and Scholl [28], and show that we achieve dramatic improvement at much higher security parameters. Gordon *et al.* implemented the binary-tree ORAM and reported a breakeven point of 8MB with a block size $D = 512$ bits, and 2^{-14} failure probability. Our breakeven point is 8KB with a block size $D = 512$ bits, and at 2^{-80} failure probability. Keller and Scholl [28] implement an optimized Path ORAM algorithm, and report a breakeven point of 9.77KB with a block size $D = 40$ bits, and a 2^{-20} failure probability. We achieve 1.3KB breakeven point at $D = 40$ bits, and with 2^{-80} failure probability.

5.1 Implementation over Garbled Circuits

Since our work, Circuit ORAM has now been implemented and provided as the default ORAM implementation in the ObliVM secure computation framework [2, 34]. Liu *et al.* [2, 34] report some Circuit ORAM performance numbers in the ObliVM framework. We report more detailed performance numbers of Circuit ORAM atop the ObliVM framework.

Running time. We tested the performance of Circuit ORAM under different network bandwidth configurations. Figure 4a and Figure 4b report the wallclock running time obtained, and the performance breakdown. To maximize performance, we apply different optimizations for these different settings. When the bandwidth is 1Gbps (Figure 4a), we use Garbled Row Reduction [41] and Free XOR [30]. When the bandwidth is 20Mbps, we use the halfgate optimization [60] instead.



(a) **Bandwidth=1Gbps.** Garbled Row Reduction [41] and Free XOR [30] are used.

(b) **Bandwidth=20Mbps.** Halfgate [60] is used.

Figure 4: **Runtime breakdown for Circuit ORAM under different bandwidth configurations.** In each pair of bars, the *left* is for the *garbler* while the *right* is for the *evaluator*. Performance numbers are measured based on the ObliVM framework provided by Liu *et al.* [34]. The plots correspond to a data size $D = 32$ bits and $\delta = 2^{-80}$ failure probability.

Interpreting the performance numbers. First, the present implementation of the ObliVM framework does not exploit AES-NI instructions to speed up garbling. We expect a noticeable speedup for the computation time when hardware AES is employed. Second, the present ObliVM uses a Java-based implementation. Therefore, the timing measurements are subject to the artifacts of Java.

In Figure 4a, when the bandwidth is 1Gbps, we observe that the garbler's garbling I/O is negligible. The evaluator's garbling I/O is higher – and most of this stems from I/O synchronization cost since in this case the bandwidth is ample. When the bandwidth is 20Mbps (Figure 4b) the garbler spends noticeable time on transmitting garbled circuits to the evaluator (I/O for Garbling). Since garbling is never blocked waiting for the evaluator, most of this cost is network transmission cost. For the OT cost, presently ObliVM does not employ the known cache optimizations [4] (which are difficult to realize in a Java-based implementation). We therefore expect significant savings in performance when ObliVM transitions to a C-based implementation adopting state-of-the-art cache optimizations for OT, and hardware AES-NI features.

Appendices Appendices are supplied in an online full version available at `https://eprint.iacr.org/2014/672.pdf`.

Acknowledgments. This research was funded in part by NSF grants CNS-1314857, CNS-1453634, CNS-1518765, CNS-1514261, a subcontract from the DARPA PROCEED program, a Sloan Research Fellowship, a Google Faculty Research Award and a grant from Hong Kong RGC under the contract HKU719312E. We gratefully acknowledge Bill Gasarch, Mor Harchol-Balter, Dov Gordon, Clyde Kruskal, Kartik Nayak, Charalampos Papamanthou, and abhi shelat for helpful discussions.

6. REFERENCES

[1] `https://eprint.iacr.org/2014/672.pdf`.

[2] `http://www.oblivm.com`.

[3] D. Apon, J. Katz, E. Shi, and A. Thiruvengadam. Verifiable oblivious storage. In *PKC*. 2014.

[4] G. Asharov, Y. Lindell, T. Schneider, and M. Zohner. More efficient oblivious transfer and extensions for faster secure computation. In *CCS*, 2013.

[5] P. Beame and W. Machmouchi. Making branching programs oblivious requires superlogarithmic overhead. In *CCC*, 2011.

[6] D. Boneh, D. Mazieres, and R. A. Popa. Remote oblivious storage: Making oblivious RAM practical. `http://dspace.mit.edu/bitstream/handle/1721.1/62006/MIT-CSAIL-TR-2011-018.pdf`, 2011.

[7] K.-M. Chung, Z. Liu, and R. Pass. Statistically-secure oram with $\tilde{O}(\log^2 n)$ overhead. In *Asiacrypt*, 2014.

[8] I. Damgård, S. Meldgaard, and J. B. Nielsen. Perfectly secure oblivious RAM without random oracles. In *TCC*, 2011.

[9] J. Dautrich, E. Stefanov, and E. Shi. Burst oram: Minimizing oram response times for bursty access patterns. In *23rd USENIX Security Symposium (USENIX Security 14)*, pages 749–764, San Diego, CA, Aug. 2014. USENIX Association.

[10] S. Devadas, M. van Dijk, C. W. Fletcher, L. Ren, E. Shi, and D. Wichs. Onion ORAM: A constant bandwidth oram without FHE, 2015.

[11] D. Dubhashi and D. Ranjan. Balls and bins: a study in negative dependence. *Random Struct. Algorithms*, 13:99–124, September 1998.

[12] C. W. Fletcher, M. v. Dijk, and S. Devadas. A secure processor architecture for encrypted computation on untrusted programs. In *STC*, 2012.

[13] C. W. Fletcher, L. Ren, A. Kwon, M. van Dijk, E. Stefanov, and S. Devadas. RAW Path ORAM: A low-latency, low-area hardware ORAM controller with integrity verification. *IACR Cryptology ePrint Archive*, 2014.

[14] C. W. Fletcher, L. Ren, X. Yu, M. van Dijk, O. Khan, and S. Devadas. Suppressing the oblivious RAM timing channel while making information leakage and program efficiency trade-offs. In *HPCA*, pages 213–224, 2014.

[15] C. Gentry, K. A. Goldman, S. Halevi, C. S. Jutla, M. Raykova, and D. Wichs. Optimizing ORAM and using

it efficiently for secure computation. In *Privacy Enhancing Technologies Symposium (PETS)*, 2013.

[16] C. Gentry, S. Halevi, C. Jutla, and M. Raykova. Private database access with he-over-oram architecture. Cryptology ePrint Archive, Report 2014/345, 2014. `http://eprint.iacr.org/`.

[17] O. Goldreich. Towards a theory of software protection and simulation by oblivious RAMs. In *STOC*, 1987.

[18] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game. In *STOC*, 1987.

[19] O. Goldreich and R. Ostrovsky. Software protection and simulation on oblivious RAMs. *J. ACM*, 1996.

[20] M. T. Goodrich. Zig-zag sort: A simple deterministic data-oblivious sorting algorithm running in O(N Log N) time. In *STOC*, 2014.

[21] M. T. Goodrich and M. Mitzenmacher. Privacy-preserving access of outsourced data via oblivious RAM simulation. In *ICALP*, 2011.

[22] M. T. Goodrich, M. Mitzenmacher, O. Ohrimenko, and R. Tamassia. Oblivious RAM simulation with efficient worst-case access overhead. In *CCSW*, 2011.

[23] M. T. Goodrich, M. Mitzenmacher, O. Ohrimenko, and R. Tamassia. Practical oblivious storage. In *CODASPY*, 2012.

[24] M. T. Goodrich, M. Mitzenmacher, O. Ohrimenko, and R. Tamassia. Privacy-preserving group data access via stateless oblivious RAM simulation. In *SODA*, 2012.

[25] S. D. Gordon, J. Katz, V. Kolesnikov, F. Krell, T. Malkin, M. Raykova, and Y. Vahlis. Secure two-party computation in sublinear (amortized) time. In *CCS*, 2012.

[26] M. Harchol-Balter. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*. Performance Modeling and Design of Computer Systems: Queueing Theory in Action. Cambridge University Press, 2013.

[27] J. Hsu and P. Burke. Behavior of tandem buffers with geometric input and Markovian output. In *IEEE Transactions on Communications. v24*, pages 358–361, 1976.

[28] M. Keller and P. Scholl. Efficient, oblivious data structures for mpc. In *ASIACRYPT*. 2014.

[29] D. G. Kendall. Stochastic processes occurring in the theory of queues and their analysis by the method of the imbedded markov chain. *The Annals of Mathematical Statistics*, 1953.

[30] V. Kolesnikov and T. Schneider. Improved Garbled Circuit: Free XOR Gates and Applications. In *ICALP*, 2008.

[31] C. P. Kruskal, M. Snir, and A. Weiss. The distribution of waiting times in clocked multistage interconnection networks. *IEEE Trans. Computers*, 37(11):1337–1352, 1988.

[32] E. Kushilevitz, S. Lu, and R. Ostrovsky. On the (in)security of hash-based oblivious RAM and a new balancing scheme. In *SODA*, 2012.

[33] C. Liu, Y. Huang, E. Shi, J. Katz, and M. Hicks. Automating efficient ram-model secure computation. In *IEEE S & P*. IEEE Computer Society, 2014.

[34] C. Liu, X. S. Wang, K. Nayak, Y. Huang, and E. Shi. ObliVM: A Generic, Customizable, and Reusable Secure Computation Architecture. S & P, 2015.

[35] J. R. Lorch, B. Parno, J. W. Mickens, M. Raykova, and J. Schiffman. Shroud: Ensuring private access to large-scale data in the data center. *FAST*, 2013.

[36] S. Lu and R. Ostrovsky. Distributed oblivious ram for secure two-party computation. In *TCC*, 2013.

[37] M. Maas, E. Love, E. Stefanov, M. Tiwari, E. Shi, K. Asanovic, J. Kubiatowicz, and D. Song. Phantom: Practical oblivious computation in a secure processor. In *CCS*, 2013.

[38] T. Mayberry, E.-O. Blass, and A. H. Chan. Efficient private file retrieval by combining oram and pir. 2014.

[39] J. C. Mitchell and J. Zimmerman. Data-Oblivious Data Structures. In *STACS*, 2014.

[40] T. Moataz, T. Mayberry, E.-O. Blass, and A. H. Chan. Resizable tree-based oblivious ram, 2015.

[41] M. Naor, B. Pinkas, and R. Sumner. Privacy preserving auctions and mechanism design. In *EC*, 1999.

[42] R. Ostrovsky. Efficient computation on oblivious RAMs. In *STOC*, 1990.

[43] R. Ostrovsky and V. Shoup. Private information storage. In *STOC*, 1997.

[44] B. Pinkas and T. Reinman. Oblivious RAM revisited. In *CRYPTO*, 2010.

[45] N. Pippenger and M. J. Fischer. Relations among complexity measures. *J. ACM*, 26(2), Apr. 1979.

[46] L. Ren, C. W. Fletcher, A. Kwon, E. Stefanov, E. Shi, M. van Dijk, and S. Devadas. Constants count: Practical improvements to oblivious ram. `http://eprint.iacr.org/2014/997/`, 2014.

[47] L. Ren, X. Yu, C. W. Fletcher, M. van Dijk, and S. Devadas. Design space exploration and optimization of path oblivious RAM in secure processors. In *ISCA*, pages 571–582, 2013.

[48] E. Shi, T.-H. H. Chan, E. Stefanov, and M. Li. Oblivious RAM with $O((\log N)^3)$ worst-case cost. In *ASIACRYPT*, 2011.

[49] E. Stefanov and E. Shi. Multi-cloud oblivious storage. In *ACM Conference on Computer and Communications Security (CCS)*, 2013.

[50] E. Stefanov and E. Shi. Oblivistore: High performance oblivious cloud storage. In *IEEE Symposium on Security and Privacy (S & P)*, 2013.

[51] E. Stefanov, E. Shi, and D. Song. Towards practical oblivious RAM. In *NDSS*, 2012.

[52] E. Stefanov, M. van Dijk, E. Shi, T.-H. H. Chan, C. Fletcher, L. Ren, X. Yu, and S. Devadas. Path ORAM: an extremely simple oblivious ram protocol. Cryptology ePrint Archive, Report 2013/280, previous version published on CCS, 2013.

[53] X. S. Wang, Y. Huang, T.-H. H. Chan, A. Shelat, and E. Shi. Scoram: Oblivious ram for secure computation. In *CCS*, 2014.

[54] P. Williams and R. Sion. Usable PIR. In *NDSS*, 2008.

[55] P. Williams and R. Sion. Single round access privacy on outsourced storage. In *CCS*, 2012.

[56] P. Williams and R. Sion. SR-ORAM: Single round-trip oblivious ram. In *CCS*, 2012.

[57] P. Williams, R. Sion, and B. Carbunar. Building castles out of mud: Practical access pattern privacy and correctness on untrusted storage. In *CCS*, 2008.

[58] A. C.-C. Yao. Protocols for secure computations (extended abstract). In *FOCS*, 1982.

[59] S. Zahur and D. Evans. Circuit structures for improving efficiency of security and privacy tools. In *S & P*, 2013.

[60] S. Zahur, M. Rosulek, and D. Evans. Two halves make a whole: Reducing data transfer in garbled circuits using half gates. In *EUROCRYPT*, 2015.